```
            r := mload(add(sig, 32))
            s := mload(add(sig, 64))
            v := byte(0, mload(add(sig, 96)))
        }
        return (v, r, s);
    }

    function recoverSigner(bytes32 message, bytes memory sig)              ❻
        internal pure returns (address)
    {
        (uint8 v, bytes32 r, bytes32 s) = splitSignature(sig);
        return ecrecover(message, v, r, s);
    }

    function prefixed(bytes32 hash) internal pure returns (bytes32){       ❻
        return keccak256(abi.encodePacked("\x19Ethereum Signed
                                                Message:\n32", hash));
    }
}
```

❶ Addresses of the organizer and worker: endpoints of micropayment channel

❷ Address of organizer in the signedMessage validated

❸ claimPayment pays worker if conditions are met

❹ Balance is a predefined attribute of an account address.

❺ Smart contract self-destructs after sending the balance to organizer

❻ Functions for recovery of signer from claim message

The smart contract has taken care of the on-chain operations of opening a channel (1), paying the claim (5), and then closing the channel (6). There is significant work left for the off-chain part, the most important of which is signing the micropayment when the worker completes verification of bins of collected garbage. The signing of the micropayment message is the responsibility of the off-chain part or the `MPC-app` module of the Dapp.

WHY DESTRUCT THE SMART CONTRACT?

Consider on-chain operation 6 in figure 7.7 and the corresponding `self-destruct(sender)` code in the `claimPayment()` function. Do you wonder why the smart contract is destructed after the payment is transferred? Imagine somebody collecting recyclable bottles and depositing them in a